Retrieval and RAG: Grounding Models in Sources

FRIDAY AI Education — Episode 014 · Printable Companion Paper

Abstract

A large language model is a reasoning engine with a memory problem. It is fluent, fast, and able to recombine what it learned during training — but it cannot tell you what happened last week, it cannot quote your company's actual contract, and when it does not know something it will often produce a confident answer anyway.
Retrieval-Augmented Generation (RAG) is the standard fix: before the model writes its answer, a retrieval step fetches relevant passages from a source you control, and the model is instructed to answer *from those passages* — citing them, bounded by them, and ideally admitting when they do not contain the answer. This paper explains the mechanism without equations, walks through where it pays off in a company (knowledge bases, fact recall, wiki Q&A, due diligence), and is honest about where it breaks. The goal is not to make you an ML engineer. It is to let you reason about retrieval the way you reason about a hire: what it's good for, what it can't be trusted with, and what governance it needs.

1. Why this matters before you build anything

Most of the value an AI system will create inside a company is not "the model is smart." It is "the model is *grounded* in our stuff." The difference between a chatbot and an operating layer is whether its answers are anchored to sources someone can check.

Consider the failure you are trying to avoid. You ask a model, "What did we promise this customer in the renewal terms?" A raw model — one with no access to your documents — will answer in the register of a plausible contract. The sentences will be grammatical, the structure will be lawyerly, and the content will be invented. This is not a bug you can prompt away. The model is doing exactly what it was built to do: produce the most likely next words. It was never given the contract.

RAG changes the question the model is answering. Instead of "what is a plausible renewal term," it becomes "given *these three paragraphs from the actual signed agreement*, what was promised." That is a question a language model is genuinely good at. You have moved the hard part — finding the right paragraphs — out of the model's head and into a system you can inspect, audit, and correct.

This is the whole strategic point. A founder does not need the model to *know* everything. A founder needs a system where (a) the authoritative sources live somewhere governed, (b) the right slice is fetched on demand, and (c) the model's answer is visibly tied back to those sources. That is a system you can put in front of an investor, a regulator, or a new hire. A clever-but-ungrounded model is not.

2. The core concept in plain language

Retrieval-Augmented Generation is a two-step pattern: retrieve, then generate.

- **Retrieve.** Take the user's question. Search a collection of documents you trust. Pull back the handful of passages most relevant to the question.
- **Generate.** Hand those passages to the language model along with the question, and instruct it to answer *using the passages*, citing which ones it used.

That is the entire idea. Everything else is engineering detail in service of those two steps. The term comes from Lewis et al. (2020), who formalized the architecture, but the intuition predates the paper: if you want a fluent system to be *correct* about specific facts, don't make it memorize the facts — let it look them up at answer time.

The reason this is powerful is a clean division of labor:

Capability	Lives in the model	Lives in retrieval
Language fluency, reasoning, summarizing	✓	
Knowing *your* specific facts		✓
Staying current (this week's data)		✓
Being correctable without retraining		✓
Producing citations	(formats them)	(supplies the sources)

The model is the reasoning and generation engine. Retrieval is the memory and the source of truth. Keeping these separate is what makes the system maintainable: when a fact changes, you edit a document, not a neural network.

3. The mechanism underneath

You do not need the math, but you do need an accurate mental model, because the failure modes live in the mechanism. Here is what actually happens.

3.1 Turning text into something searchable

You cannot search a large body of text the way you search a single sentence with Ctrl-F, because the words a user types rarely match the words in the document. Someone asks about "time off policy"; the document says "paid leave entitlement." Keyword search misses it. RAG systems solve this with **embeddings**.

An embedding is a list of numbers that represents the *meaning* of a chunk of text — its position in a space where passages with similar meaning sit close together. "Time off policy" and "paid leave entitlement" land near each other even though they share no words. This nearness is called **semantic similarity** or **vector distance**. It is not geographic, and it is not truth — two passages being close means they are *about similar things*, not that either one is correct. Hold onto that distinction; it matters in Section 7.

The process, end to end:

1. **Chunk.** Break your documents into passages — a few paragraphs each. Too big and you retrieve noise; too small and you lose context.
2. **Embed.** Convert each chunk into its embedding (its meaning-coordinates) and store them in a database built for similarity search (a vector store).
3. **Index.** Organize those embeddings so you can find the nearest ones quickly, even across millions of chunks.

This is done once, ahead of time, and refreshed as documents change.

3.2 Answering a question

When a question arrives:

1. **Embed the question** the same way you embedded the chunks.
2. **Search** the vector store for the chunks whose meaning sits closest to the question's meaning. Pull back the top few — typically three to ten.
3. **Assemble a prompt** that contains: the instruction ("answer using only the sources below; cite them; if the answer isn't here, say so"), the retrieved chunks, and the original question.
4. **Generate.** The model writes an answer constrained by what it was handed.

The retrieved passages are placed into the model's **context window** — its working attention for this one request. They are not learned or remembered after the request ends. This is why RAG is correctable in real time: change the document, re-embed it, and the next answer reflects the change immediately. No retraining, no waiting.

3.3 The instruction is load-bearing

The least technical step is the one founders most often underestimate. The instruction given to the model in step 3 is what separates a grounded system from a fluent liar. A good instruction does three things: it tells the model to rely on the supplied sources, it tells the model to *cite* which source each claim came from, and — critically — it tells the model what to do when the sources don't contain the answer. We return to that last one in Section 7, because it is the difference between a system you can trust and one you can't.

4. Citations and source boundaries

Two features turn retrieval from a convenience into something you can stand behind in a serious room.

Citations mean every claim in the answer points back to the passage it came from. Not a vague "according to company policy" but a specific, checkable pointer: *this sentence rests on that paragraph of that document.* The value is not decoration. A citation converts an assertion into a *verifiable* assertion. The reader — an investor, an auditor, a new employee — can click through and confirm. This is the mechanism by which an AI answer becomes admissible in a high-stakes conversation. An uncited answer asks for trust; a cited answer offers proof.

Source boundaries mean the system is explicitly scoped to a defined corpus, and it knows the edge of that corpus. A due-diligence assistant should answer from the data room and *only* the data room. If asked something outside it, the correct behavior is to say the answer is not in scope — not to reach into the model's general training and improvise. Boundaries are what let you make promises about an AI system. "This assistant answers only from our published research" is a claim you can audit. "This assistant answers from whatever it happens to know" is not a claim at all.

Together these give you the property that makes organizational AI defensible: **traceability**. Every answer has a provenance, and the set of possible sources is known and bounded. That is the posture you want before you let any of this touch a customer, an investor, or a regulator.

5. Freshness

Training is a snapshot. A model knows the world roughly up to the point its training data was collected, and not a day past. Ask it about anything that happened after — a pricing change you made yesterday, this morning's metrics, last week's board resolution — and it has no genuine basis to answer. It will sometimes answer anyway, which is worse than silence.

Retrieval is how a system stays current without being rebuilt. Because the documents are searched at answer time, the system is exactly as fresh as the corpus behind it. Update the document, re-embed it, and the next answer is current. The model never changed; the *sources* did.

This reframes a question founders get wrong. The question is rarely "how smart is the model." It is "how fresh and well-maintained is the corpus it reads from." A brilliant model reading a stale wiki gives confident, current-sounding, wrong answers. A modest model reading a clean, current corpus gives useful ones. **In a RAG system, the corpus is the product.** Your operational discipline around keeping sources accurate matters more than which model you picked.

6. Where this shows up in a company

Four concrete patterns, in rough order of how good a first wedge they are.

6.1 Public-content knowledge base — the strong first wedge

Take a person's or a company's entire public body of work — talks, essays, interviews, posts, papers — and make it searchable, reusable, and *cited*. A question goes in; out comes an answer grounded in the actual material, with pointers to the specific source for each claim.

This is the cleanest place to start, for reasons that are strategic, not just technical. The corpus is public, so there is no confidentiality risk while you build the muscle. The ground truth is checkable by anyone, so quality is easy to evaluate. And the output is immediately useful: a body of work that was previously a pile of links becomes something you can interrogate, quote accurately, and operate on. You learn the full retrieve-cite-bound loop on low-stakes data before you point it at anything sensitive.

6.2 Fact recall over a memory substrate

A persistent store of facts an operating layer should remember — decisions made, commitments given, the current state of ongoing work — that an assistant can recall on demand instead of reconstructing from scratch each session. RAG is the recall mechanism: the question is matched against stored facts, the relevant ones are retrieved, and the answer is grounded in them. The discipline of *citing which stored fact* an answer rests on is what keeps an assistant's memory honest rather than a second source of plausible fiction.

6.3 Company wiki Q&A

The internal case of 6.1. Employees ask questions; the system answers from approved internal documents, with citations, scoped to what the asker is allowed to see. The payoff is that institutional knowledge stops living only in the heads of the three people who've been around longest. The new constraint is **permissions**: retrieval must respect who is allowed to see what. A retrieval system that ignores access control is a data-leak engine with a friendly interface — it will cheerfully surface the compensation spreadsheet to anyone who asks the right question. Scope

retrieval to the asker's permissions, always.

6.4 Investor due diligence packet

The highest-stakes case, and the one that most rewards getting boundaries right. An assistant answers questions about your company strictly from the curated data room: metrics, contracts, cap table, financials. Every answer cites the underlying document. Anything outside the room is explicitly out of scope. Done well, this is faster, more consistent, and more auditable than a junior team member fielding the same questions — and it never improvises a number under pressure. Done badly — without strict boundaries and without honest "not in the room" behavior — it invents a figure during diligence, which is roughly the worst moment for an AI system to be confidently wrong.

7. The main failure mode: confident wrongness, and "I don't know"

Everything above can be in place and the system can still fail in a specific, dangerous way. This section is the one to annotate.

7.1 Retrieval can fetch the wrong thing

The model's answer is only as good as what retrieval handed it. Several ways that goes wrong:

- **The answer isn't in the corpus at all.** The user asks something the documents don't cover.
- **Retrieval pulls passages that are *similar* but not *relevant*.** Remember: similarity is not truth. A chunk can sit close to the question in meaning-space while being about the wrong product line, an outdated policy, or a superficially related topic. The system retrieves it because it's *near*, not because it's *right*.
- **The right passage exists but ranks just below the cutoff,** so it never makes it into the prompt.

In all three cases, the model is now reasoning over inadequate material — and a fluent model will produce a fluent answer regardless.

7.2 The "I don't know" behavior is the whole ballgame

Here is the crux. When the retrieved passages do not contain the answer, there are two possible behaviors:

1. The system says, in effect, *"The sources don't contain this."*
2. The system fills the gap from the model's general training and answers anyway — confidently, fluently, and unmoored from any source.

Behavior 2 is the default unless you deliberately engineer behavior 1. A raw model is built to continue text plausibly; absence of evidence does not naturally translate into "I don't know." You have to instruct it to abstain, scope it to the corpus, and — this is the part teams skip — *test* that it actually abstains.

A system that reliably says "I don't know" when it doesn't know is more valuable than a system that is right slightly more often but occasionally fabricates with total confidence. The reason is trust calibration: if the system abstains honestly, you can *believe its answers*, because you know it would have abstained otherwise. If it sometimes bluffs, every answer carries an invisible asterisk, and the whole system drops back to "useful suggestion, verify manually." Honest abstention is what lets an AI answer carry weight in a serious conversation. It is a feature, not a limitation, and it is the single most important thing to verify before deployment.

7.3 This is why governance is not optional

Because confident wrongness is the native failure mode, useful organizational AI needs more than a model and a vector store. It needs:

- **Permissions** — retrieval scoped to what the asker may see.
- **Evals** (evaluations/tests) — a standing set of questions, including ones the corpus *can't* answer, to confirm the system retrieves correctly and abstains honestly. Run them whenever the corpus or the model changes.
- **Audit** — citations and logs, so any answer can be traced to its sources after the fact.
- **Human approval gates** — for anything consequential, a person reviews before the answer becomes an action.

None of this makes the model smarter. All of it makes the *system* trustworthy. That is the trade an operator should be happy to make. An agent is not a magic employee you can point at a problem; it is a component that becomes dependable only inside this scaffolding.

8. Vocabulary

- **RAG (Retrieval-Augmented Generation)** — the two-step pattern of fetching relevant source passages, then generating an answer constrained by them.
- **Retrieval** — searching a trusted collection of documents to find the passages most relevant to a question.
- **Generation** — the language model writing an answer, here constrained to the retrieved passages.
- **Embedding** — a numerical representation of a chunk of text's *meaning*, used so that passages about similar things can be found even when they share no words.
- **Semantic similarity / vector distance** — how close two pieces of text are in meaning-space. Closeness signals *aboutness*, not correctness. Not geographic.
- **Vector store** — a database that holds embeddings and finds the nearest ones to a query quickly.
- **Chunk** — a passage of a document, sized so it carries enough context to be useful but not so much that it adds noise.
- **Context window** — the model's working attention for a single request; retrieved passages are placed here and not retained afterward.
- **Citation** — an explicit pointer from a claim in the answer to the source passage it rests on; turns an assertion into a checkable one.
- **Source boundary** — the defined, known edge of the corpus a system is allowed to answer from.
- **Freshness** — how current the answers are, which in a RAG system depends on the corpus, not the model.
- **"I don't know" behavior** — the engineered, tested ability to abstain when the sources don't contain the answer, rather than improvising.
- **Evals** — evaluations: a standing test set used to verify retrieval quality and honest abstention. (Pronounced like "evaluations," not "evils.")

9. Reading guide

You do not need to read both end to end before the next walk. Read in this order, with this purpose:

Pinecone — *What is RAG?* (https://www.pinecone.io/learn/retrieval-augmented-generation/)

Start here. It is the practitioner's explanation of the mechanism in Section 3 — embeddings, vector search, assembling the prompt — in plain operational language. Read it to make the retrieve-then-generate pipeline concrete. *Bias note:* Pinecone sells a vector database, so the framing naturally centers that component. The concepts are sound; just register that the vendor's interest is in the part they sell.

Lewis et al. (2020) — *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*
(https://arxiv.org/abs/2005.11401)

This is the source of the term and the architecture. You do not need the equations or the benchmark tables. Read the **abstract and introduction**, and skim the figure showing the retrieve-then-generate flow. The single idea to extract: separating a parametric memory (what the model learned) from a non-parametric memory (the documents it retrieves) makes the system more accurate on fact-heavy tasks *and* updatable without retraining. That is the whole thesis, and it is the intellectual foundation for everything in this paper.

Fifteen minutes on the first and twenty skimming the second is enough.

10. Walking questions

Carry these on the next walkcast. They are for thinking, not for homework.

1. Which part of RAG could you now explain cleanly to a smart non-technical person — the retrieve step, the generate step, citations, or boundaries?
2. Which part still feels blurry? (If it's embeddings and semantic similarity, that's the expected blurry spot — sit with it.)
3. Where does grounding already show up, or where *should* it, in the systems you're building — and is the corpus behind it actually maintained, or quietly going stale?
4. Name one bounded workflow where you could test retrieval safely this month. The public-content knowledge base is the obvious candidate: low stakes, checkable ground truth, and it teaches the full retrieve-cite-bound loop before you ever point it at anything confidential.
5. For that workflow: what does "I don't know" look like, and how would you *test* that the system actually says it instead of bluffing?

One-sentence takeaway

RAG makes a fluent model *trustworthy* by forcing it to answer from sources you control, cite what it used, stay within known boundaries, and admit when the answer isn't there — which means the real work isn't the model, it's the corpus and the governance around it.